STEWARDSHIP HARNESSES: MAINTAINING EXECUTABLE INTELLIGENCE ACROSS AGENT LOOPS

Josh Rosen
ThruWire, Inc.

Seth Rosen
ThruWire, Inc.

June 11, 2026

ABSTRACT

Multi-agent AI systems increasingly consist of agents with their own task loops: they plan, call tools, delegate work, and produce outputs through interaction with external systems. Interoperability helps those agents communicate, but communication alone does not maintain the shared structure future agents inherit. In long-lived, dependency-bearing work, locally successful loops can accumulate cross-agent drift: stale assumptions, inconsistent artifacts, duplicated procedures, incompatible context selection, and final outputs that no longer agree with maintained upstream state.

We hypothesize that such systems need shared executable intelligence that these agent loops can consume and collectively steward: durable, versioned procedures and artifacts that future agents can run against. A shared stewardship harness is an MCP-accessible execution control plane that maintains current artifacts, executable context patterns, supersession, authority, dependency-aware invalidation, affectedness, and stewardship integration. Outer loops use this structure when it applies, ask the harness to execute relevant structure with task inputs, consume the resulting artifacts, and propose updates when their work reveals gaps.

The contribution is a systems abstraction and reference architecture, not an empirical benchmark. We define the shared stewardship harness and executable context patterns, distinguish agent interoperability from maintained executable state, outline an MCP-accessible harness interface, and propose evaluation criteria for testing cross-agent drift and maintained-state quality.

1 Introduction

Agent loops are the familiar operating pattern of contemporary agents: they explore, call tools, recover from partial failure, and adapt to a task [8, 9, 11, 10, 12, 13]. We use *outer loop* as a local architectural term for this task-directed agent loop when it surrounds calls into the shared stewardship harness. The term is meant to distinguish the agent's open-ended task loop from the inner graph nodes, materialization steps, validation checks, and recomputation paths executed by the harness. The same flexibility becomes a systems problem when many loops work over long-lived shared state. A locally successful loop may answer a task while leaving the maintained criteria artifact stale, invent a better procedure but preserve it only in a transcript, or retrieve a different context bundle than a peer loop. Across many agents, local task success can compound global drift.

We call this *cross-agent drift*: loss of coherence as agents assemble different context, preserve different assumptions, update different artifacts, and leave authority unresolved. Agent communication is not maintained executable shared state. Agents may exchange messages fluently while still disagreeing about which artifacts are current, which procedures should run, which outputs are stale, and which changes affect downstream work.

This paper proposes a shared stewardship harness for long-lived, dependency-bearing multi-agent systems: an MCP-accessible execution control plane that maintains executable context patterns, intermediate artifacts, supersession, current-state resolution, and affectedness. *Executable intelligence* is the part of shared state that future agents can run against: context patterns, artifact contracts, authority structure, dependency structure, validation steps, and material-

ization behavior learned over time. Outer loops are both consumers and maintainers of this structure. A stewardship-compliant loop does not first produce an independent final answer and then ask how to clean up. It asks at task selection time what structure already exists, uses the harness to materialize and execute relevant context patterns or graph regions, consumes the intermediate artifacts produced by that execution, and routes new observations back into maintained structure as they arise.

This framing builds on prior ThruWire work on execution lineage and first-class intermediate artifacts [1, 2]. The present paper asks how many outer loops can maintain the executable structure future loops inherit. A2A-style interoperability helps agents find and delegate to other agents [3]; MCP provides a schema-described tool and context interface to the harness [4, 5, 6, 7]. LLM-maintained wikis and Git repositories are also valuable shared-state mechanisms, but the question here is what provides executable context patterns, authority, supersession, affectedness, and stewardship validation.

The paper makes four contributions:

- We define outer-loop stewardship as the protocol expectation that existing agent loops consume current shared structure when it applies, select and run the relevant harness-managed block or graph region with task inputs, and keep the shared execution environment aligned with what the task discovers, or justify why no update was needed.
- We define the shared stewardship harness as an execution control plane for maintained intelligence across many outer loops.
- We introduce executable context patterns as versioned procedures for assembling, omitting, generating, and validating the right artifacts and constraints for a class of task.
- We provide a reference architecture, MCP-accessible harness interface, lifecycle model, limitations, and evaluation agenda for measuring cross-agent drift and stewardship quality.

The claims are deliberately bounded. We do not claim that a shared harness replaces A2A, replaces communication, eliminates drift, guarantees better final answers, or belongs in every task. We propose an architecture and evaluation agenda for long-lived AI work where future agents depend on maintained structure.

2 Motivating Example: Company Intelligence and Strategy Analysis

Consider company intelligence executed over months by multiple humans and agents: founder research, funding updates, technical novelty assessments, competitive-risk memos, criteria edits, and customer-implication reports. Each task may appear successful while the shared state degrades. One loop treats a stale market map as current; another uses a superseded rubric; another updates a final memo but leaves the technical novelty artifact unchanged.

The failure is not absence of text or memory. It is absence of maintained executable structure: a durable procedure for deciding what context to materialize, which artifacts are authoritative, which versions are superseded, which downstream artifacts depend on a criteria artifact, and what must be reviewed after change. The procedure can also encode how the task should think, not only what it should load. For company intelligence, the harness can require a fan-out of sealed perspective artifacts: technical read, founder and incentive read, company-stage read, market-category read, product-pattern read, narrative read, competitive-threat read, and an explicit noise or overfit check. It can require at least one artifact to argue the contrary position: why the target may not matter, why vocabulary overlap is not substance, what not to overreact to, and what evidence would change the decision.

With a shared stewardship harness, the risk-assessment loop asks for an executable context pattern:

> For a competitor-risk assessment, bind the target company, reference company, peer set, market category, and stage band; load the current company model, founder and stage artifact, funding artifact, technical novelty artifact, competitive landscape, strategic priorities, open questions, and current risk criteria; omit stale market-map versions and superseded rubrics; materialize independent technical, market, product, narrative, threat, and noise-check artifacts; require the final judgment to weigh strongest evidence against strongest counterargument.

The harness materializes current artifacts and runtime slots, then executes the reusable perspective structure as part of producing the final brief. If the loop discovers that technical novelty needs different criteria for research-heavy and services-heavy companies, stewardship means routing that observation into a context-pattern or criteria change for harness-mediated execution, not merely mentioning the distinction in the final memo. If the contrary read shows that the target was mostly noise, the loop should still leave useful maintained state through the harness: a no-action rationale, future revisit conditions, or a deduplication note that helps future agents avoid rediscovering the same non-signal. Figure 1 sketches the architecture.

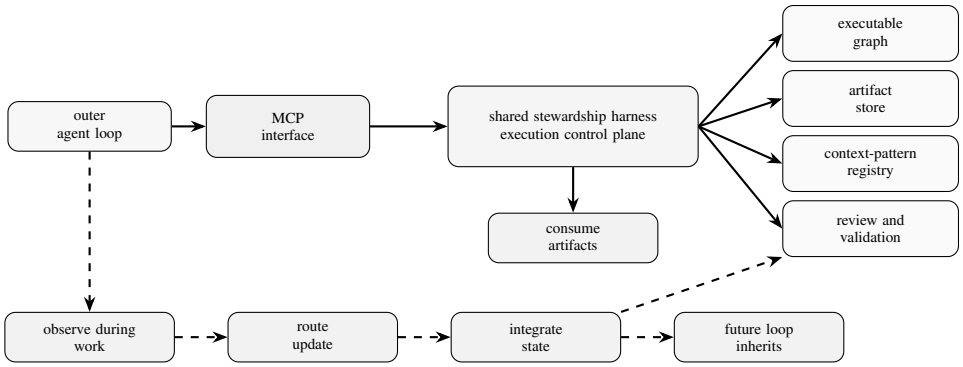

Figure 1: Outer loops use shared executable structure through MCP while the task is being performed. The harness materializes context, executes graph regions, produces or registers artifacts, and integrates observations or proposed changes so future loops inherit improved maintained state.

2.1 Concrete Lifecycle Trace

A compact task trace makes the architecture operational:

1. **Input task.** An outer loop receives: "Reassess competitive risk for Company X in the infrastructure-software market."
2. **Selection.** Through MCP, the loop searches or selects the current compiled structure for `competitor_risk_assessment`: a context pattern, graph region, output contract, and relevant artifact roles.
3. **Materialization.** The harness resolves current company, funding, technical-novelty, competitive-landscape, and risk-criteria artifacts; excludes stale maps and superseded rubrics; binds `peer_set`, `market_category`, and `stage_band`; and returns a context bundle plus output contract.
4. **Perspective execution.** The harness executes or materializes independent lens artifacts: technical fit, product implications, strategic implications, competitive threat, narrative implications, and a contrary noise or overfit check.
5. **Task execution.** The loop consumes those artifacts, performs the open-ended judgment, and asks the harness to produce or register an updated risk-assessment artifact that states both the strongest evidence and strongest counterargument.
6. **Stewardship update.** If the task reveals a reusable criteria distinction, missing lens, or better contrary check, the loop records the observation and proposes a criteria or context-pattern change.
7. **Affectedness and integration.** The harness applies or routes the proposed change through review or publication steps, marks affected downstream artifacts stale, and validates that the task execution consumed current artifacts, produced the expected output through the artifact contract, and routed the pattern change for review if required.

3 Background and Related Work

3.1 Interoperability and Tool/Context Access

A2A defines an interoperability layer for independent agents: discovery, task delegation, messages, artifacts, streaming, and protocol bindings [3]. MCP connects AI applications to external systems through schema-described tools, resources, resource templates, and prompts [4, 5, 6, 7]. In this paper, A2A routes work among agents, MCP exposes the harness to outer loops, and the shared stewardship harness supplies execution-control semantics.

3.2 Harnesses and Procedural Memory

Agent harness work treats the harness around a model as an object of study. Natural-Language Agent Harnesses externalize control logic as editable executable artifacts with contracts, durable artifacts, and adapters [15]; related surveys and systems emphasize planning, tool use, memory, verification, state, and control [16, 17, 18]. Procedural-memory and workflow-memory systems preserve reusable guidance: procedures, trajectories, decompositions, and experiences that help later agents plan or act more effectively [19, 20, 21, 22, 23, 24]. The stewardship harness is

different because it is stateful over the work products themselves. It maintains intermediate and final artifacts, graph or block execution state, run records, lineage, and context patterns as shared executable state. Procedural memory can suggest how a future loop should proceed; the harness determines what maintained artifacts and executable structures that loop should run against, update, or leave unchanged.

3.3 Shared Knowledge and Repository Mechanisms

LLM-maintained wikis compound synthesized knowledge over time [25, 26]. Git repositories coordinate versioned files, review, rollback, attribution, and CI hooks [27]. Knowledge graphs, vector stores, and wikis preserve shared facts, relations, notes, or retrieved memories [35]. These mechanisms may store or review harness inputs, but storage is not execution semantics. If `risk_rubric.md` changes from `risk_criteria:v1` to `risk_criteria:v2`, Git can represent the patch and CI can run tests; the harness determines whether the new rubric is current for a role and scope, which context patterns use it, and which downstream artifacts consumed the old rubric.

Mechanism	Primary durable object	Good at	Usually lacks by default
LLM-maintained wiki	Markdown pages, summaries, links, indexes	Shared accumulated knowledge and synthesis	Runtime slots, artifact authority, invalidation, recomputation
Git repository	Versioned files, commits, branches, diffs, pull requests	Shared review, history, rollback, CI integration	Semantic current-state resolution for LLM-native artifacts
Vector memory	Embeddings and retrieved memories	Recall and similarity-based reuse	Explicit dependency structure and supersession
Knowledge graph	Entities and relations	Structured factual and relational knowledge	Executable context procedures and affectedness
Stewardship harness	Context patterns, artifacts, graph state, review structure, runtime records	Maintained executable intelligence across loops	Requires schema, governance, and stewardship discipline

Table 1: Adjacent shared-state mechanisms can support a stewardship harness, but do not by themselves provide execution-control semantics.

3.4 Blackboards, Workflow DAGs, Provenance, and Multi-Agent Frameworks

Blackboard systems identify the value of a shared workspace where multiple specialists contribute partial results [28]. They are a useful historical analogy for agent systems because they separate local specialist behavior from shared intermediate state. The stewardship harness differs by making the shared state executable, versioned, and dependency-bearing rather than only a posted working memory.

Workflow systems such as Dryad, Spark, Airflow, and dbt use dependency structure, materialized intermediates, and lineage-aware execution [29, 30, 31, 32]; provenance research studies why and where outputs arise from prior state [33, 34]. These systems are closest to the paper's execution-lineage foundation, but they usually coordinate data or software workflows rather than agent loops that discover and revise context patterns while performing open-ended work.

Multi-agent frameworks such as CAMEL, AutoGen, MetaGPT, Voyager, and Magentic-One organize agents into roles, conversations, planners, and tool users [10, 11, 12, 13, 14]. They help structure agent teams and task execution. The stewardship harness addresses a different layer: the maintained executable state those agents consume, update, and validate across many runs.

4 Problem Formulation

4.1 Outer Loops

An *outer loop* is the paper's term for the agentic task loop surrounding the shared stewardship harness. In common agent terminology, this is simply an agent loop: it plans, calls tools, reasons, delegates, observes, and completes a task. We treat it as the accountable unit because it consumes task context, acts, and decides what observations, inputs, or proposed structural changes should be returned to the shared execution environment.

Let $C = \{c_1, \ldots, c_n\}$ be clients or outer loops. Each loop c_i receives a task objective τ_i and a materialized context bundle b_i. It performs open-ended work and may produce messages, tool calls, draft judgments, observations, or proposed mutations. Maintained artifacts are created or registered by the harness according to artifact contracts and compiled review or publication structure. Without a shared stewardship protocol, the loop's natural optimization target is isolated task completion:

$$c_i : (\tau_i, b_i) \to y_i \tag{1}$$

where y_i is an isolated task output. Isolated completion does not imply maintained shared state.

4.2 Architectural Objective

The architectural objective is to reduce drift accumulation across repeated tasks while preserving task-completion performance. In long-lived workloads with dependency-bearing artifacts, executable stewardship should improve maintained-state quality: fewer stale current artifacts, fewer duplicated procedures, more consistent context materialization, and more accurate affectedness after changes. This is structural, not a claim that the harness makes models smarter. Future evaluations should measure both drift reduction and the added latency, review burden, and proposal noise.

4.3 Cross-Agent Drift

Cross-agent drift occurs when many locally successful loops cause shared state to become less coherent. We distinguish five forms:

- **Context drift:** different loops assemble incompatible context bundles for similar tasks.
- **Artifact drift:** final outputs are updated while upstream intermediate artifacts remain stale.
- **Procedure drift:** loops evolve incompatible task procedures or criteria.
- **Authority drift:** multiple artifacts are treated as current without explicit resolution.
- **Evaluation drift:** loops optimize isolated task outputs without maintaining reusable structure.

These failures can occur even when every loop appears competent. The missing element is a maintained execution environment that makes shared state legible, current, and improvable.

4.4 Observed Failure Patterns

The failure modes above are not experimental results in this paper. They are illustrative patterns that motivate the proposed architecture and that future evaluations should measure directly.

Failure pattern	Typical behavior in loop-centric systems
Artifact drift	A final report is updated, but the criteria artifact or intermediate analysis it depends on remains stale.
Procedure drift	Two agents independently evolve different evaluation rubrics for similar tasks and preserve them only in isolated outputs or transcripts.
Authority drift	Multiple summaries, rubrics, or analyses are treated as current because no role/scope-specific resolver selects the active artifact.
Context drift	Similar tasks materialize different context bundles because each loop performs ad hoc retrieval or uses a different memory fragment.
Evaluation drift	Agents optimize the visible final answer but do not update reusable procedures, validation steps, or dependency structure.
Stewardship noise	Many agents propose low-value observations, criteria edits, or pattern changes, increasing review burden without improving maintained state.

Table 2: Illustrative failure patterns that a stewardship harness is intended to make visible and measurable.

4.5 Why Shared Memory and A2A Are Insufficient Alone

Shared memory can improve recall, and A2A can route work to a capable agent, but neither by itself defines artifact authority, supersession, context materialization, or downstream affectedness. In long-lived settings, loops need a layer where they can inspect and improve shared executable state.

5 Shared Stewardship Harness

5.1 Definition

We define a shared stewardship harness as:

$$\mathcal{H} = (G, A, P, R, C, \Pi) \tag{2}$$

where:

- G is executable graph structure: blocks, steps, tasks, and dependency edges.
- A is maintained artifact state: intermediate and final artifacts, versions, statuses, lineage, authority scope, and supersession records.
- P is the set of executable context patterns.
- R is the set of runtime records: executions, inputs, slots, resolved dependencies, validations, and materialization events.
- C is the set of clients or outer loops accessing the harness through MCP.
- Π is policy: permissions, approval, supersession, invalidation, validation, conflict handling, and review requirements.

The harness is shared because many outer loops use it. It is a stewardship harness because it maintains the executable knowledge layer future loops inherit. In consumer mode, a loop resolves the relevant pattern, artifacts, graph nodes, compiled version, and slots, then asks the harness to run the selected block or graph region with task inputs. In steward mode, the loop returns observations, evidence, and proposed changes to criteria, dependencies, context patterns, or validation steps; the harness applies, routes, or rejects those changes through its compiled review and publication structure.

5.2 Execution Control Plane

The *execution control plane* determines what is current, stale, reusable, recomputable, reviewable, or accepted into maintained state. It answers questions such as:

- Which artifact is current for this role and scope?
- Which downstream artifacts become stale after this supersession?
- Which context pattern should be used for this goal type?
- Which existing graph region or artifact contract should this task execute against?
- Which proposed graph mutation violates a dependency or authority boundary?
- Which affected subgraph should be recomputed, reviewed, or left unchanged?

5.3 Artifacts, Current State, and Supersession

Following artifact-centric state, artifacts in A are typed, addressable, versioned, dependency-aware, and consumable by downstream computation [2]. Each artifact has a role, scope, lineage, status, and authority boundary. A superseding artifact can replace an earlier artifact as current for a role and scope while preserving audit history.

Current-state resolution is therefore a maintained operation:

$$\text{current}(role, scope, t) \rightarrow a_k \tag{3}$$

where a_k is the artifact that the execution control plane considers active at time t according to Π. If no artifact is current, or if multiple candidates conflict, the harness should surface that condition rather than letting each outer loop infer authority locally.

The hard case is authority resolution. Because maintained artifacts are produced or registered by harness execution, conflicts should not be modeled as arbitrary outer loops writing competing current artifacts. They arise when two harness runs, graph versions, criteria revisions, or review decisions could each make a different artifact current, or when an artifact is current for one scope but not another. Implementations may use human review, trust tiers, confidence thresholds, ownership metadata, or deterministic resolution steps such as preferring approved runs over draft runs. If the compiled review or publication structure cannot resolve the conflict, artifact reads and current-state queries should surface a structured unresolved or missing state rather than a silent guess.

5.4 Invalidation and Propagation

If artifact a is superseded by a', every downstream artifact whose execution identity depended on a may require invalidation, recomputation, or review. The harness should support:

- exact preservation of unaffected branches,
- stale marking before recomputation,
- selective recomputation of affected descendants,
- review nodes for high-authority or high-risk updates,
- publication of superseding artifacts when recomputation completes.

This preserves the execution-lineage distinction between final output quality and maintained-state quality.

6 Minimum Viable Stewardship Harness

The smallest useful harness should include:

- an artifact registry with role, scope, version, status, lineage, and authority metadata,
- a current-state resolver for `current(role, scope, t)`,
- a context-pattern registry with versioned selection, exclusion, slot, and output-contract declarations,
- a materialization function that binds inputs and slots and returns a context bundle with provenance,
- supersession and stale-marking operations,
- validation or integration steps for task completion,
- optional human review nodes for high-authority changes.

This minimum version excludes general agent-to-agent communication, broad social negotiation, full workflow orchestration, automatic proof that all dependencies were found, and automatic acceptance of agent-proposed pattern changes. Even a small harness should make current state explicit, make context materialization executable, record supersession, and support stewardship integration throughout task execution.

7 Executable Context Patterns

7.1 Definition

A *context pattern* is an executable, versioned procedure for assembling the right context for a class of task. It is not just a prompt template and not just retrieval. A context pattern declares which artifacts, sources, roles, constraints, inputs, and slots should be loaded, omitted, generated, refreshed, or validated at runtime.

We define a context pattern as:

$$p = (g, I, S, Q, M, E, V, O) \tag{4}$$

where:

- g is the goal type, such as competitor-risk assessment.
- I is the input schema.
- S is the slot schema for runtime bindings.
- Q is the set of selection declarations for artifacts, sources, and roles.
- M is the set of generation or materialization steps.
- E is the set of exclusion declarations.
- V is the set of validation steps or review nodes.
- O is the output contract.

A pattern is executable because it accepts runtime inputs and slots, resolves current artifacts, may materialize generated intermediates, and returns a bounded context bundle with provenance.

7.2 Example

For a company-risk assessment, a context pattern might be:

- **Goal type:** competitor-risk assessment.
- **Inputs:** target company, assessment date, requested risk lens.
- **Slots:** peer set, market category, prior thesis, stage band.
- **Selection declarations:** load current company profile, founder/stage artifact, funding artifact, technical novelty artifact, comparable-company pattern artifact, and current risk criteria.
- **Exclusion declarations:** omit stale market-map versions, superseded risk rubrics, unrelated prior summaries, and artifacts outside the target market category.
- **Generation steps:** materialize peer-set comparison and risk-criteria checklist at runtime.
- **Validation steps:** resolve current state for each authoritative artifact; flag missing technical novelty state.
- **Output contract:** produce or register a risk assessment artifact with role, scope, lineage, confidence, supersession target if any, and downstream dependents.

This pattern encodes context precision: load these artifacts but not those artifacts; generate this intermediate only after slots are bound; reject or review the task if the authoritative criteria artifact is missing. The pattern is learned structure, not merely a longer prompt.

7.3 How Patterns Improve

Outer loops improve context patterns when they discover that a pattern is incomplete, overloaded, stale, too broad, too narrow, or reusable in a stronger form. The company-risk example in Section 2 illustrates the expected behavior: the loop turns a procedural discovery into a criteria update, slot, branch, or validation step rather than preserving it only in the final memo.

Pattern improvement should be reviewable. Bad patterns can propagate mistakes by making future loops confidently load the wrong context. The harness therefore needs versioning, evidence, validation steps, rollback or supersession, and review or publication structure for pattern changes.

8 MCP as the Harness Interface

8.1 Harness Surface

MCP is a natural interface because it lets heterogeneous outer loops access the same shared harness without sharing one internal architecture. The important point is not that the harness exposes a large collection of isolated tools. The MCP surface is a control surface for a maintained system: projects, blocks, folders, compiled versions, executable graph state, runs, event streams, artifacts, and review or publication steps. Ordinary-looking operations such as reading a block, compiling a version, running a block, polling run events, and reading an artifact become stewardship operations when they participate in compiled structure that carries lineage, authority, slots, cached context, and publication behavior.

The exact tool names are implementation-specific. A harness may expose semantic operations directly, or realize them through lower-level project, block, version, run, and artifact operations. The useful surface groups capabilities by function:

- **Discovery and selection:** list projects, folders, blocks, context patterns, graph nodes, and available artifacts; search compiled structure when the loop does not know the exact block to run.
- **Authoring and mutation:** read, create, edit, move, rename, or delete raw blocks and folders through the harness's authoring surface; propose graph or context-pattern changes as structured mutations rather than transcript notes.
- **Compilation and versioning:** compile raw structure into an immutable executable version; resolve the current compiled version; preserve prior versions for audit, rollback, and affectedness analysis.
- **Execution and materialization:** run a selected block or graph region with runtime inputs and slot bindings; materialize the context bundle, intermediate artifacts, final artifacts, and provenance implied by that run.
- **Observation and integration:** stream or poll run events, inspect produced artifacts, record observations, run review or publication blocks, mark affected state stale, and execute the stewardship-integration structure for the task.

The design requirement is that a loop can discover or select an existing executable graph region, run it with task-specific inputs and slots, consume the resulting context and artifacts, and integrate observations or proposed edits into maintained structure through schema-described operations. In some implementations, high-level materialization or integration operations may be exposed directly. In others, those semantics emerge from compiling a graph, running selected blocks, resolving artifacts, inspecting run events, and executing review or publication steps encoded in the graph. Both are harness designs, not merely tool collections.

8.2 Harness Operation Sketches

The following sketches are illustrative and use generic operations for a compiled-block harness: search compiled blocks, inspect the current DAG, read current artifacts, execute blocks, poll run events, edit raw blocks, and compile a new version. They should not be read as a separate rule engine or as a claim that every implementation exposes these exact names. Checks and constraints are expressed through executable blocks, compiled graph structure, artifact contracts, run events, and review or publication steps.

```
select_current_structure(goal, inputs):
  matches = search_compiled_blocks(query=goal)
  dag = get_current_dag()
  return choose_block_or_graph_region(matches, dag, inputs)

consume_current_artifact(block_id, inputs, slot_bindings):
  result = read_current_artifact(block_id, inputs, slot_bindings)
  if result.status == "success": return result.artifact
  return result  // e.g., missing artifact or slot-binding guidance

run_structured_work(block_id, inputs, slot_bindings):
  accepted = run_block(block_id, inputs, slot_bindings)
  events = poll_run_events(accepted.run_id)
  return {run_id: accepted.run_id, events: events}

inspect_outputs(block_id, inputs, slot_bindings):
  artifact = read_current_artifact(block_id, inputs, slot_bindings)
  artifacts = list_current_artifacts()
  return {artifact: artifact, project_artifacts: artifacts}

steward_structure(block_path, revised_block):
  existing = read_raw_block(block_path)
  upsert_raw_block(block_path, revised_block)
  compiled = compile_project_version()
  return {previous: existing.revision, version: compiled.version_id}
```

8.3 Relationship to A2A

A2A routes work among agents; MCP exposes the shared harness to those agents; the harness maintains execution semantics through its project, graph, version, run, artifact, and review/publication machinery. Table 3 summarizes the boundary.

Layer	Solves	Does not solve by itself
A2A	Agent discovery, delegation, task exchange, and message-level interoperability.	Shared maintained intelligence, artifact authority, or dependency-aware invalidation.
MCP	Schema-described access to harness operations, resources, runs, artifacts, and context.	The maintained-state semantics unless backed by a harness.
Shared stewardship harness	Current state, durable artifacts, executable context patterns, compiled versions, runs, supersession, and propagation.	General communication, social negotiation, or all agent-to-agent coordination.

Table 3: A2A, MCP, and the shared stewardship harness operate at complementary layers.

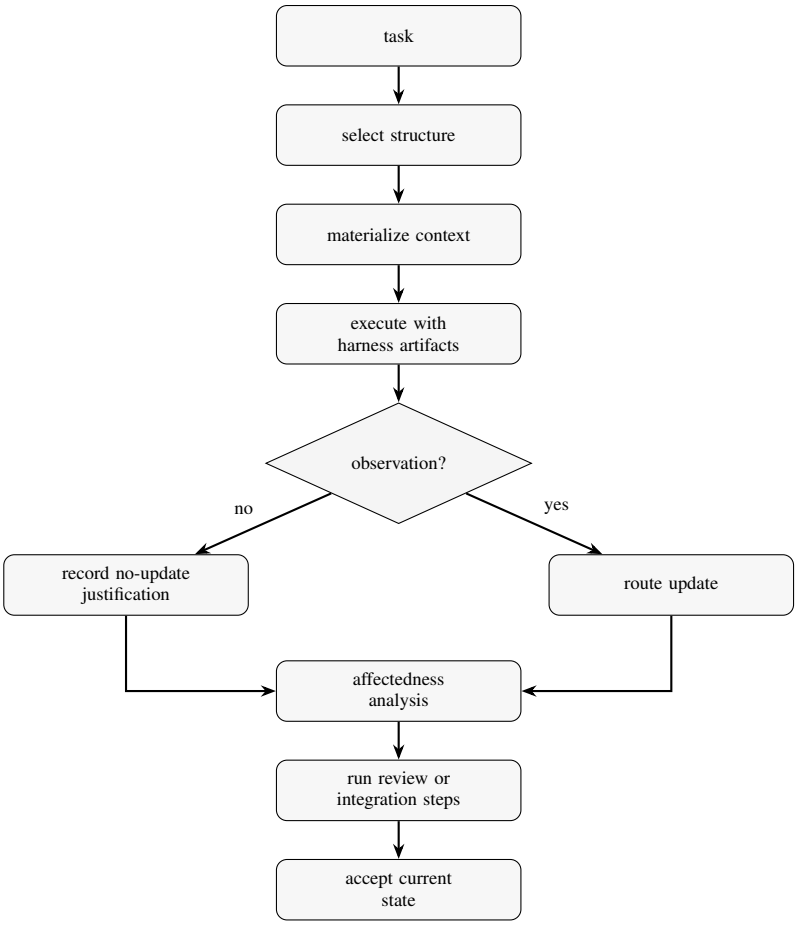

Figure 2: Stewardship is integrated into task execution. The outer loop selects and runs harness structure early, consumes generated artifacts while doing the work, and routes observations through affectedness, review, and integration steps before current state is accepted.

The two layers can compose. An A2A client may delegate a task to a specialized agent. That agent may then use MCP to materialize context from the shared harness, execute the task with harness-produced intermediates, ask the harness to produce or register artifacts, and integrate stewardship updates before returning an A2A artifact. A2A helps route the task to the right agent. Context patterns help the selected agent inherit the right structure.

9 Integrated Stewardship Lifecycle

At task selection time, the outer loop asks whether the harness already contains relevant executable structure: a context pattern, graph region, artifact contract, validation step, or recomputation path. If so, the loop asks the harness to run the selected structure with its inputs, parameters, and slots, then consumes the resulting artifacts, generated intermediates, validation warnings, and provenance. The resulting answer is therefore not purely local; it is produced by collaboration between the outer loop's open-ended reasoning and the harness's maintained executable structure.

During execution, the loop remains flexible: it may call tools, delegate subtasks, inspect artifacts, and explore. The harness remains active as the task proceeds by materializing intermediates, executing selected blocks, recording run events, resolving artifacts, and making missing context, overloaded context, stale artifacts, ambiguous authority, or reusable procedural discoveries available as candidate improvements rather than side notes.

As work proceeds, the loop and harness should integrate evidence and proposed changes into maintained state:

- updated intermediate artifacts to be produced or registered through harness execution,
- final artifact supersessions to be accepted through review or publication structure,

- graph dependency edits,
- context-pattern improvements,
- validation-step changes,
- stale markings or recomputation requests,
- explicit "no shared update needed" justifications.

A task that bypasses this integrated process can still be locally useful, but the produced answer may not be aligned with maintained state. The harness validates whether the task execution consumed current artifacts, left stale state, exposed unresolved authority conflicts, affected downstream dependents, or discovered reusable procedure improvements that should be routed into maintained structure. Future loops then inherit the revised shared execution environment.

10 Evaluation Agenda

This paper does not report a new experiment and should not be read as empirical validation. It proposes a systems abstraction, terminology, design requirements, and reference architecture. Future work should test whether shared stewardship harnesses reduce cross-agent drift and improve maintained-state quality under multi-loop workloads.

10.1 Metrics

The metrics should evaluate maintained execution state, not only final prose quality:

- **Context-pattern quality:** reuse rate, improvement acceptance rate, context precision and recall, duplicate procedure creation, and future-agent lift from prior pattern improvements.
- **Artifact-state quality:** stale artifact rate, authority conflicts, stale or superseded artifact consumption, invalidation correctness, propagation completeness, and cross-agent consistency.
- **Stewardship behavior:** useful update rate, no-update-needed justifications, human review burden, proposal acceptance and rejection, stewardship-noise rate, time to identify affected subgraphs, and drift accumulation over N tasks.
- **Wiki and repository comparison:** stale page or file consumption, rediscovered procedures despite shared content, updates that become executable context-pattern changes, time to assess downstream impact, file changes without affectedness metadata, and future-loop use of the correct current artifact.

10.2 Proposed Benchmark Design

A small benchmark can make the hypothesis falsifiable without requiring a full product-scale deployment. One design is a repeated company-intelligence or research-synthesis workload with evolving criteria. Each trial would run a sequence of related tasks and introduce a legitimate criteria change midway through the sequence.

The comparison conditions should include:

- independent agent loops with ad hoc retrieval,
- a shared LLM-maintained wiki,
- a shared Git repository of markdown pages, prompts, policies, and tests,
- shared memory or vector retrieval,
- A2A-style delegation without a shared stewardship harness,
- MCP-mediated stewardship over maintained artifacts and context patterns.

The benchmark should measure stale artifact rate, context consistency across agents, duplicate procedure creation, affectedness accuracy, review burden, and future-agent reuse. The expected result is not assumed. A shared wiki or repository may perform well on recall and reuse; the proposed harness should be judged specifically on maintained-state metrics such as current artifact resolution, invalidation correctness, propagation completeness, context-pattern consistency, and stewardship integration.

Other task families include strategy planning with evolving criteria, codebase maintenance where requirements and verification artifacts must remain aligned, and multi-agent competitive analysis where context patterns determine which static and generated artifacts enter context. Future experiments should avoid claiming that the harness makes models smarter or guarantees better final answers. The appropriate hypotheses are structural:

- fewer completed tasks leave stale authoritative artifacts,
- affected subgraphs are identified more quickly and accurately,
- future loops assemble more consistent context bundles,
- useful procedural improvements are reused rather than rediscovered,
- final outputs agree more often with maintained upstream state.

11 Limitations

The proposed architecture has important limits.

- **Latency and coordination overhead.** Context matching, current-state resolution, validation, and review can slow down work that would otherwise be completed locally.
- **Not every task justifies the machinery.** Exploratory, one-off, low-stakes, or rapidly changing tasks may be better served by ad hoc agent work, ordinary communication, or simple shared notes.
- **Bad structure and pattern drift.** A flawed context pattern, incorrect authority decision, or overfit procedure may make future loops consistently wrong.
- **False invalidations and missed dependencies.** Overbroad affectedness can create unnecessary recomputation and review; omitted dependencies can leave stale artifacts active.
- **Stewardship noise and governance bottlenecks.** Many agents may propose edits, pattern diffs, stale markings, and invalidations. Without thresholds, deduplication, batching, ownership metadata, and review queues, stewardship activity can become noise or concentrate into slow approval queues.
- **Private model reasoning should not be persisted.** The harness maintains artifacts, context patterns, validations, and provenance, not hidden chain-of-thought.
- **Communication remains necessary.** A2A, chat, review comments, and human discussion still matter; the harness does not replace them.
- **Evaluation remains future work.** The paper provides an agenda and architecture, not benchmark results.

12 Conclusion

As agent systems scale from one loop to many long-lived, dependency-bearing loops, the central problem may become not only how agents talk, delegate, or remember, but what shared executable structure they can consume and collectively maintain. LLM-maintained wikis and shared repositories show that agents can accumulate knowledge and coordinate over durable files. The shared stewardship harness builds on that intuition but changes the maintained object: what future loops should execute, which artifacts are current, what has become stale, and how task execution updates the shared execution environment.

We proposed the shared stewardship harness as an MCP-accessible execution control plane for multi-agent AI systems with long-lived shared state. A2A lets agents find and communicate with other agents; MCP lets outer loops access the harness; the harness maintains artifacts, context patterns, authority, supersession, invalidation, affectedness, and stewardship integration.

The hypothesis behind stewardship harnesses is that multi-agent systems become more capable not only by adding more agents, but by giving those agents shared executable structure that compounds across tasks. When each loop can consume maintained structure and improve it during execution, the system can accumulate usable procedure, context, and artifact state rather than merely accumulating messages or outputs. This may allow long-lived agent teams to approach problems whose complexity exceeds what any single loop can reliably reconstruct from scratch.

This remains a proposal and evaluation agenda: multi-agent systems should be evaluated not only by whether a task was completed, but by whether the next agent inherits better maintained executable state.

References

[1] Josh Rosen and Seth Rosen. From Agent Loops to Deterministic Graphs: Execution Lineage for Reproducible AI-Native Work. arXiv:2605.06365, 2026.

[2] Josh Rosen and Seth Rosen. Intermediate Artifacts as First-Class Citizens: A Data Model for Durable Intermediate Artifacts in Agentic Systems. arXiv:2605.12087, 2026.

[3] Agent2Agent Project. Agent2Agent (A2A) Protocol Specification, version 1.0.0. `https://a2a-protocol.org/latest/specification`, accessed June 11, 2026.

[4] Model Context Protocol. What is the Model Context Protocol (MCP)? `https://modelcontextprotocol.io/docs/getting-started/intro`, accessed June 11, 2026.

[5] Model Context Protocol. Specification 2025-06-18: Tools. `https://modelcontextprotocol.io/specification/2025-06-18/server/tools`, accessed June 11, 2026.

[6] Model Context Protocol. Specification 2025-06-18: Resources. `https://modelcontextprotocol.io/specification/2025-06-18/server/resources`, accessed June 11, 2026.

[7] Model Context Protocol. Specification 2025-06-18: Prompts. `https://modelcontextprotocol.io/specification/2025-06-18/server/prompts`, accessed June 11, 2026.

[8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023.

[9] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, 2023.

[10] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. arXiv:2303.17760, 2023.

[11] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023.

[12] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. arXiv:2308.00352, 2023.

[13] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291, 2023.

[14] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. arXiv:2411.04468, 2024.

[15] Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-Language Agent Harnesses. arXiv:2603.25723, 2026.

[16] Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, Lingjie Chen, Yanjun Zhao, Ke Yang, Bingxuan Li, Cheng Qian, Gaotang Li, Xiao Lin, Zhichen Zeng, Ruizhong Qiu, Sirui Chen, Yifan Sun, Xiyuan Yang, Ruida Wang, Rui Pan, Chenyuan Yang, Dylan Zhang, Liri Fang, Zikun Cui, Yang Cao, Pan Chen, Dorothy Sun, Ren Chen, Mahesh Srinivasan, Nipun Mathur, Yinglong Xia, Hong Li, Hong Yan, Pan Lu, Lingming Zhang, Tong Zhang, Hanghang Tong, and Jingrui He. Code as Agent Harness: Toward Executable, Verifiable, and Stateful Agent Systems. arXiv:2605.18747, 2026.

[17] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. AutoHarness: Improving LLM Agents by Automatically Synthesizing a Code Harness. arXiv:2603.03329, 2026.

[18] Yuxuan Zhang, Haoyang Yu, Lanxiang Hu, Haojian Jin, and Hao Zhang. General Modular Harness for LLM Agents in Multi-Turn Gaming Environments. arXiv:2507.11633, 2025.

[19] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent Workflow Memory. arXiv:2409.07429, 2024.

[20] Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring Agent Procedural Memory. arXiv:2508.06433, 2025.

[21] Shengda Fan, Xin Cong, Yuepeng Fu, Zhong Zhang, Shuyan Zhang, Yuanwei Liu, Yesai Wu, Yankai Lin, Zhiyuan Liu, and Maosong Sun. WorkflowLLM: Enhancing Workflow Orchestration Capability of Large Language Models. arXiv:2411.05451, 2024.

[22] Dongge Han, Camille Couturier, Daniel Madrigal Diaz, Xuchao Zhang, Victor Rühle, and Saravan Rajmohan. LEGOMem: Modular Procedural Memory for Multi-Agent LLM Systems for Workflow Automation. arXiv:2510.04851, 2025.

[23] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schütze, Volker Tresp, and Yunpu Ma. Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning. arXiv:2508.19828, 2025.

[24] Luiz C. Borro, Luiz A. B. Macarini, Gordon Tindall, Michael Montero, and Adam B. Struck. Memori: A Persistent Memory Layer for Efficient, Context-Aware LLM Agents. arXiv:2603.19935, 2026.

[25] Andrej Karpathy. LLM Wiki. GitHub Gist, 2026. `https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f`, accessed June 11, 2026.

[26] Shuide Wen and Beier Ku. Knowledge Compounding: An Empirical Economic Analysis of Self-Evolving Knowledge Wikis under the Agentic ROI Framework. arXiv:2604.11243, 2026.

[27] Scott Chacon and Ben Straub. Pro Git. 2nd edition, Apress, 2014. `https://git-scm.com/book/en/v2`, accessed June 11, 2026.

[28] H. Penny Nii. Blackboard Systems, Part One: The Blackboard Model of Problem Solving and the Evolution of Blackboard Architectures. *AI Magazine*, 7(2):38–53, 1986.

[29] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. Dryad: Distributed Data-Parallel Programs from Sequential Building Blocks. In *EuroSys*, 2007.

[30] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster Computing with Working Sets. In *USENIX HotCloud*, 2010.

[31] Apache Software Foundation. Airflow Documentation: DAGs. `https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html`, accessed June 11, 2026.

[32] dbt Labs. dbt Developer Hub. `https://docs.getdbt.com/`, accessed June 11, 2026.

[33] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and Where: A Characterization of Data Provenance. In *International Conference on Database Theory*, 2001.

[34] Juliana Freire, David Koop, Emanuele Santos, and Claudio T. Silva. Provenance for Computational Tasks: A Survey. *Computing in Science & Engineering*, 10(3):11–21, 2008.

[35] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan F. Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge Graphs. *ACM Computing Surveys*, 54(4):71:1–71:37, 2021.